

PROJECTFUEL

Red de distribución de combustible

DOCUMENTO DE DESPLIEGUE

Backend + Frontend Android

Universidad Piloto de Colombia
Autor: Juan Diego Galindo

Tabla de contenido

Tabla de contenido.....	2
1. Estado actual del despliegue.....	3
2. Componentes del sistema	3
3. Requisitos previos.....	3
4. Despliegue local — Backend.....	3
4.1 Crear la base de datos	3
4.2 Configurar credenciales	3
4.3 Ejecutar el servidor	4
4.4 Verificar que el backend responde.....	4
5. Despliegue local — Frontend	4
5.1 Obtener la IP local del backend	4
5.2 Configurar la URL base.....	4
5.3 Configurar la clave de Google Maps	4
5.4 Compilar e instalar.....	5
6. Limitaciones del despliegue actual	5
7. Recomendaciones para un despliegue de producción.....	5

1. Estado actual del despliegue

A la fecha de este documento, ProjectFuel no cuenta con un despliegue en la nube ni con un dominio público. El sistema fue diseñado, construido y probado como un entorno local de desarrollo: el backend (Spring Boot) corre en la máquina del desarrollador, y la aplicación Android se conecta a él a través de la red local (LAN) mediante una dirección IP fija configurada dentro del código del cliente.

Este documento describe, en primer lugar, cómo levantar ese entorno local tal como existe hoy, y en segundo lugar, qué cambios serían necesarios para llevar el sistema a un despliegue de producción real, siguiendo el mismo nivel de exigencia aplicado en otros proyectos del portafolio.

2. Componentes del sistema

Componente	Tecnología	Repositorio
Backend	Spring Boot 3.3.4, Java, Maven, MySQL	ProjectFuel-Backend
Frontend	Android nativo (Java), Gradle	ProjectFuel-Frontend
Base de datos	MySQL 8, esquema autogenerado por Hibernate	—

3. Requisitos previos

- JDK 17 o superior y Maven (o el wrapper mvnw incluido) para el backend
- MySQL Server 8 en ejecución, accesible en localhost:3306
- Android Studio y un dispositivo físico o emulador con API 23+ para el frontend
- Backend y dispositivo Android conectados a la misma red local

4. Despliegue local — Backend

4.1 Crear la base de datos

El backend no crea la base de datos por sí mismo, únicamente las tablas dentro de ella. Antes de arrancar la aplicación es necesario crear manualmente la base de datos vacía:

```
CREATE DATABASE projectfuel;
```

4.2 Configurar credenciales

El archivo `application.properties` define la conexión a la base de datos. Por defecto usa el usuario `root` sin contraseña, apuntando a una instancia local:

```
spring.datasource.url=jdbc:mysql://localhost:3306/projectfuel
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
server.port=8080
server.address=0.0.0.0
```

Estos valores deben ajustarse a las credenciales reales del entorno donde se despliegue. `server.address=0.0.0.0` hace que el servidor escuche en todas las interfaces de red, lo cual es necesario para que un dispositivo Android en la misma LAN pueda alcanzarlo.

4.3 Ejecutar el servidor

```
git clone https://github.com/juandiegogalindo/ProjectFuel-Backend.git
cd ProjectFuel-Backend
./mvnw spring-boot:run
```

Al arrancar, Hibernate crea o actualiza automáticamente las tablas del esquema a partir de las entidades JPA (`ddl-auto=update`). El servidor queda escuchando en el puerto 8080.

4.4 Verificar que el backend responde

Desde la misma máquina, o desde otro equipo en la red, se puede confirmar que el servicio está activo consultando cualquier endpoint de solo lectura, por ejemplo la lista de combustibles, con una herramienta como Postman o simplemente desde el navegador:

```
http://<IP-de-la-máquina>:8080/combustibles
```

5. Despliegue local — Frontend

5.1 Obtener la IP local del backend

Es necesario conocer la dirección IP de la máquina donde corre el backend dentro de la red local (por ejemplo, mediante `ipconfig` en Windows o `ip addr` en Linux/Mac).

5.2 Configurar la URL base

Esa IP debe reemplazar el valor fijo que actualmente existe en el cliente Android:

```
// ApiClient.java
private static final String BASE_URL = "http://<IP-de-la-máquina>:8080/";
```

5.3 Configurar la clave de Google Maps

Antes de compilar, se debe reemplazar la clave de API de Google Maps (con Maps SDK for Android y Directions API habilitadas) en `strings.xml`, `AndroidManifest.xml`, y dentro de la llamada a `Directions` en `MapaEstacionesActivity.java`.

5.4 Compilar e instalar

```
git clone https://github.com/juandiegogalindo/ProjectFuel-Frontend.git
cd ProjectFuel-Frontend
./gradlew installDebug
```

Con el backend corriendo y el dispositivo en la misma red, la aplicación queda lista para iniciar sesión y operar con datos reales.

6. Limitaciones del despliegue actual

- No existe despliegue en la nube: el sistema solo ha sido probado en red local; no hay una instancia accesible por internet.
- IP fija no portable: la URL del backend está escrita directamente en el código del frontend, por lo que cualquier cambio de red obliga a recompilar la aplicación.
- Tráfico sin cifrar: toda la comunicación entre la app y el backend viaja por HTTP plano, habilitado explícitamente en la configuración de seguridad de red de Android.
- Credenciales en texto plano: la base de datos usa el usuario root sin contraseña, y las contraseñas de usuario de la aplicación se almacenan y se comparan sin cifrar (findByCorreoAndPassword compara el valor tal cual).
- Sin contenedores ni orquestación: no existen Dockerfile, docker-compose ni scripts de infraestructura como código en ninguno de los dos repositorios.
- Sin CI/CD: no hay pipelines de integración o despliegue continuo configurados.

7. Recomendaciones para un despliegue de producción

Si el proyecto se llevara a un entorno de producción real, se recomienda el siguiente conjunto de cambios, en orden de prioridad:

- Contenerizar el backend con Docker y desplegarlo en un servicio administrado (por ejemplo, AWS Elastic Beanstalk, ECS o un EC2 con Docker Compose, siguiendo el mismo enfoque usado en el proyecto Red Vial).
- Mover la base de datos a un servicio administrado (Amazon RDS para MySQL) con un usuario de aplicación dedicado y contraseña fuerte, distinto del usuario root.
- Sustituir ddl-auto=update por un sistema de migraciones versionado (Flyway o Liquibase) para tener control explícito sobre los cambios de esquema.
- Servir el backend detrás de HTTPS (por ejemplo, con un certificado gestionado y un dominio propio), y eliminar la excepción de tráfico en texto plano del lado de Android.
- Externalizar la URL base y las claves de API del frontend mediante variantes de build (debug/release) en lugar de valores fijos en el código.
- Incorporar hashing de contraseñas (BCrypt) en el backend y un mecanismo de autenticación basado en tokens (JWT) en lugar de la validación de credenciales en cada solicitud.
- Configurar un pipeline de CI/CD que compile, pruebe y despliegue automáticamente ambos repositorios ante cada cambio en la rama principal.